

// git

Stacked-diffs

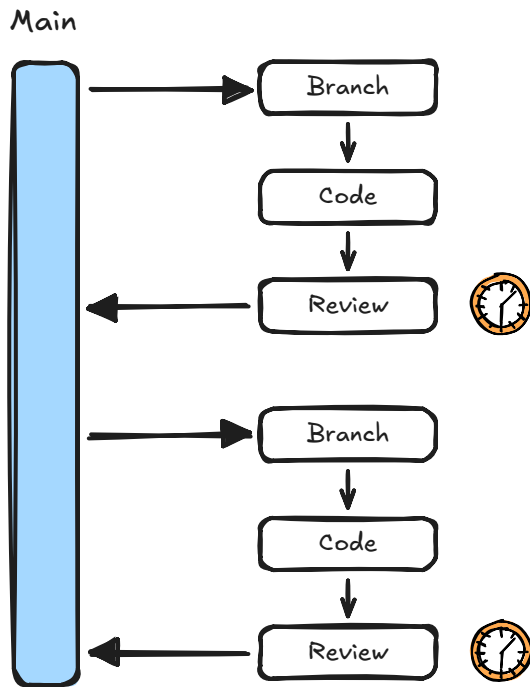
always committing to main



The problem

- Working on things in parallel can be hard
- Often have to wait for review before starting the next step
- Creating branches for every change gives a mental overhead

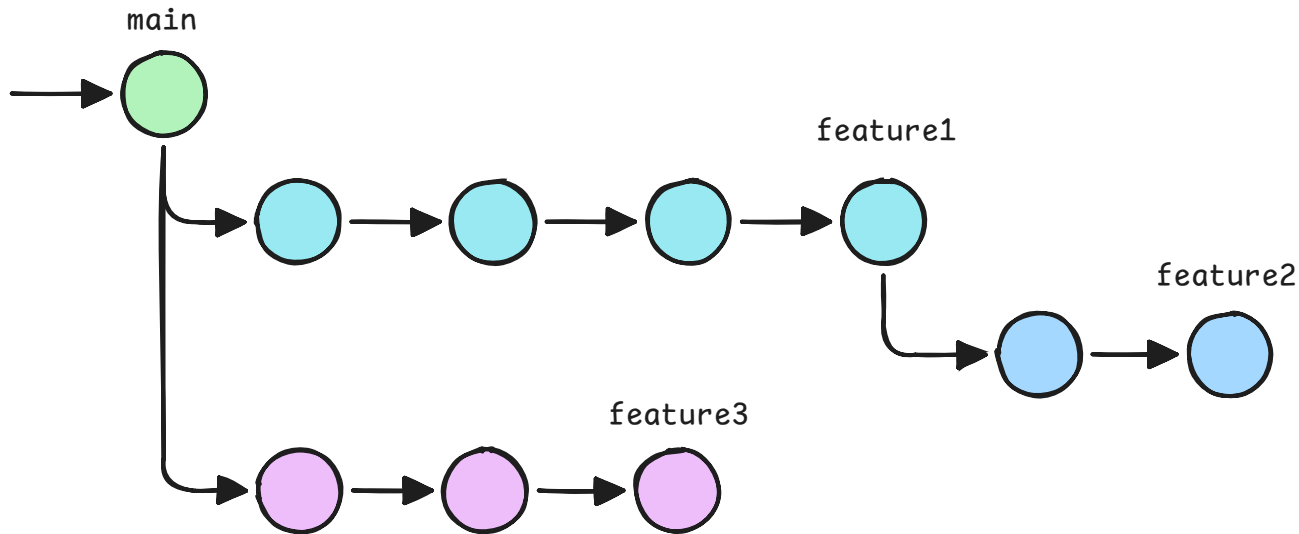
If I have to branch off a bugfix branch I will usually go and make a cup of coffee instead.



The pitch

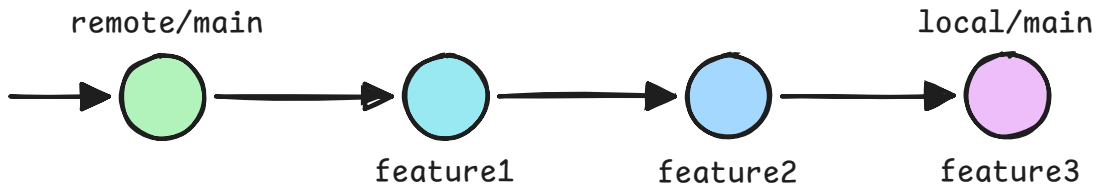
- ◆ Why do we have to do all these git things that never end up in main anyway?
- ◆ We want to just continue working, not having to setup structures to do so
- ◆ Bonus: small reviews, clean history

Traditional pull-request based approach



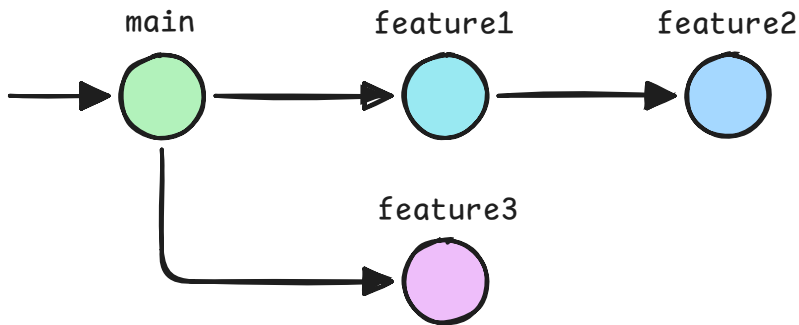
Ideal stacked-diffs based approach

Camp: "I just want to work"



Alternative stacked-diffs based approach

Camp: "graph should reflect merge order"



Stacked-diffs

- Every commit ("diff") is a unit of review
- Every commit must be able to ship on its own
 - Makes the commit history very clean → similar to squash with ff
 - Makes bisecting a valuable debugging tool
- Focus on atomic changes, many small merge requests
- Stack new work on unmerged changes, tooling handles the rest

Clean commit history

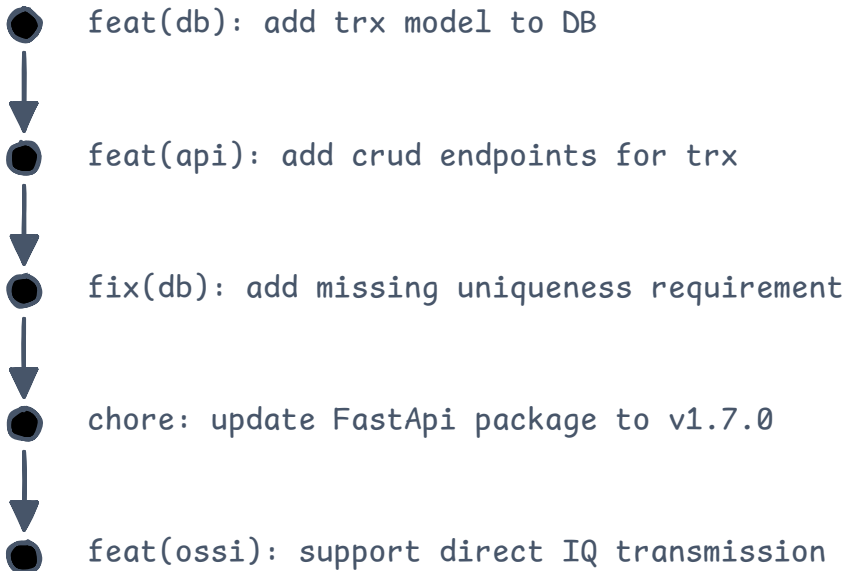
Stacked-diffs

- Each commit is a job from the start
- Review feedback incorporated by amending commit
- History is clean as a natural byproduct

Pull-requests with branches

- Write messy commits as you go
- Clean up later, which is often hard and therefore skipped
- End up with "address review comments"

Clean commit history



Note

This can be achieved with squash-merges and fast-forward, however, this will change the SHA of the main branch commits, making rebasing and cherry-picking a lot harder than they need to be.

Tooling is necessary

- Converting "diffs" to merge-requests
- Automatically rebasing an entire stack
- Run build + lint + tests on every commit in a stack
- Squashing fixup commits
- Syncing "landed" diffs with local master

This is where everything falls apart for me

More experimentation needed

- The promises of stacked-diffs seem nice to me
- I have only barely tried to use the workflow, but it's easier to try as a team
- I hope that this might be a way to guide our ambition of breaking up larger tasks into smaller ones
- There will be more reviews, but I hope that the work will in total be quicker as an MR should be a defined unit

Further reading

- <https://jg.gg/2018/09/29/stacked-diffs-versus-pull-requests/>
The "original" blog post explaining the workflow
- <https://www.stacking.dev>